Linux Disk Commands

A complete, practical guide to checking, managing, and troubleshooting disks and partitions in Linux — built for AWS / DevOps learners.

Covers: df, du, lsblk, fdisk, parted, mkfs, mount/umount, blkid, fsck, and a quick-reference cheat sheet.

1. Why Disk Commands Matter

On any Linux server — including EC2 instances — disk commands let you check how much storage is used, find out what's eating up space, view and manage partitions, format new volumes, and mount/unmount filesystems. These are core Linux administration skills and come up constantly in DevOps work: attaching an EBS volume, resizing a partition, or debugging a 'disk full' alert.

Disk commands generally fall into three groups:

- **Monitoring** — df, du, lsblk, blkid, iostat (how much space / which disks exist)
- **Partitioning & formatting** — fdisk, parted, mkfs (create and prepare storage)
- **Mounting & integrity** — mount, umount, fsck (attach storage to the filesystem tree, check for errors)

2. df - Disk Free (space usage per filesystem)

Purpose: Shows how much disk space is used and available on mounted filesystems.

Basic syntax

```
df [OPTIONS] [FILE]
```

Common options

Option	Meaning
-h	Human-readable sizes (KB/MB/GB instead of raw blocks)
-T	Show filesystem type (ext4, xfs, etc.)
-i	Show inode usage instead of block usage
-a	Include pseudo/duplicate filesystems normally hidden
--total	Add a grand total row

Examples

```
df -h
df -hT
df -h /var
df -i
```

Note: df -h is the single most common command for a quick "am I running out of disk space?" check.

3. du - Disk Usage (space used by files/directories)

Purpose: Shows how much space specific files or directories are consuming. While df reports space at the filesystem level, du drills into folders to find what is using the space.

Basic syntax

```
du [OPTIONS] [PATH]
```

Common options

Option	Meaning

-h	Human-readable sizes
-s	Summary — total for the directory only, not every subfolder
-a	Show sizes for files as well as directories
--max-depth=N	Limit how many directory levels deep to show
-c	Show a grand total at the end

Examples

```
du -sh /var/log
du -h --max-depth=1 /home
du -ah /etc | sort -rh | head -10 # 10 biggest items in /etc
```

*Note: du -sh * inside a directory quickly shows which sub-item is the space hog — very useful for finding what filled the disk.*

4. lsblk - List Block Devices

Purpose: Lists all block devices (disks, partitions, volumes) attached to the system in a tree view — the fastest way to see disk/partition layout, including on a freshly attached EBS volume.

Basic syntax

```
lsblk [OPTIONS]
```

Common options

Option	Meaning
-f	Also show filesystem type, label, and UUID
-a	Show empty/unmounted devices too
-o	Choose custom output columns, e.g. -o NAME,SIZE,TYPE,MOUNTPOINT

Example output

```
$ lsblk
NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINT
xvda 202:0 0 8G 0 disk
■■xvda1 202:1 0 8G 0 part /
xvdf 202:80 0 20G 0 disk
```

Note: In the example, xvdf is a new EBS volume with no partition or mount yet — a common state right after attaching storage.

5. fdisk - Partition Table Editor (MBR-focused)

Purpose: Creates, deletes, and lists partitions on a disk. Works interactively — you run it against a disk device, then use single-letter commands inside its prompt.

Basic syntax

```
sudo fdisk [OPTIONS] DEVICE
sudo fdisk -l # list all partitions on all disks
```

Inside the interactive fdisk prompt

Key	Action
m	Show help menu
p	Print the current partition table
n	Create a new partition
d	Delete a partition
t	Change a partition's type
w	Write changes to disk and exit
q	Quit without saving

Example workflow — partitioning a new disk

```
sudo fdisk -l # find the new disk, e.g. /dev/xvdf
sudo fdisk /dev/xvdf # enter interactive mode
Command (m for help): n # new partition
Command (m for help): p # (accept defaults for primary/size as needed)
Command (m for help): w # write and exit
```

Note: fdisk changes are NOT applied until you press 'w'. Pressing 'q' safely exits without saving.

6. parted - Advanced Partitioning (GPT-friendly)

Purpose: Like fdisk but supports GPT partition tables and disks larger than 2TB, and can run in a single non-interactive command — handy for scripts.

```
sudo parted /dev/xvdf print # show partition table
sudo parted /dev/xvdf mklabel gpt # create a GPT table
sudo parted /dev/xvdf mkpart primary ext4 0% 100% # one partition, full disk
```

7. mkfs - Make Filesystem (format a partition)

Purpose: Formats a partition with a filesystem type (ext4, xfs, etc.) so it can actually store files. You always run this AFTER partitioning, BEFORE mounting.

Common variants

Command	Filesystem created
mkfs.ext4 /dev/xvdf1	ext4 (very common default on Linux)
mkfs.xfs /dev/xvdf1	XFS (default on many RHEL/Amazon Linux systems)
mkfs.vfat /dev/xvdf1	FAT32 (for compatibility, e.g. USB drives)

Note: Formatting ERASES all data on that partition. Double-check the device name before running mkfs.

8. mount / umount - Attach and Detach Filesystems

Purpose: mount attaches a filesystem (partition) to a directory (the "mount point") so its files become accessible; umount detaches it safely.

Basic syntax

```
sudo mount /dev/xvdf1 /mnt/data # mount a partition
mount | grep xvdf1 # confirm it's mounted
sudo umount /mnt/data # unmount when done
```

Persisting across reboot: /etc/fstab

A mount done this way disappears after reboot. To make it permanent, add a line to /etc/fstab with the device (or UUID), mount point, filesystem type, and options:

```
UUID=xxxx-xxxx /mnt/data ext4 defaults,nofail 0 2
```

Note: Get the UUID with 'blkid' (see next section) rather than typing the device name, since device names like /dev/xvdf can shift.

9. blkid - Identify Block Devices

Purpose: Shows the UUID, filesystem type, and label of block devices — needed for /etc/fstab entries.

```
sudo blkid
sudo blkid /dev/xvdf1
```

```
/dev/xvdf1: UUID="3a1e..." TYPE="ext4" PARTUUID="..."
```

10. fsck - Filesystem Check and Repair

Purpose: Checks a filesystem for errors (e.g. after an unclean shutdown) and can repair them.

```
sudo umount /dev/xvdf1 # must be unmounted first
sudo fsck /dev/xvdf1
```

Note: Never run fsck on a mounted filesystem — always unmount it first, or you risk data corruption.

11. Putting It Together: Attaching a New Disk (EBS Example)

A typical end-to-end flow when a new volume is attached to a Linux server:

- **lsblk** — confirm the new disk is visible (e.g. /dev/xvdf, no partitions yet)
- **fdisk /dev/xvdf** (or parted) — create a partition
- **mkfs.ext4 /dev/xvdf1** — format the new partition
- **mkdir /mnt/data && mount /dev/xvdf1 /mnt/data** — mount it
- **blkid /dev/xvdf1** — get its UUID
- Add a line to **/etc/fstab** using that UUID — so it survives reboot
- **df -h** — confirm it shows up with correct space

12. Quick Reference Cheat Sheet

Command	What it does
df -h	Free/used space per mounted filesystem
du -sh DIR	Total size of a directory
lsblk -f	Tree of disks/partitions with filesystem info
fdisk -l	List partition tables (MBR-focused, interactive editor)
parted DEVICE print	List partitions (GPT-friendly, scriptable)
mkfs.ext4 DEVICE	Format a partition with ext4
mount DEVICE DIR	Attach a filesystem to a directory
umount DIR	Detach a mounted filesystem
blkid	Show UUID / filesystem type of devices
fsck DEVICE	Check/repair a filesystem (must be unmounted)

Tip: practice these on a spare EC2 instance with an extra EBS volume attached — nothing teaches disk commands faster than actually partitioning, formatting, and mounting a real volume.